

Object-Oriented Programming

How a program is organized around **objects** — bundles of data and the behavior that acts on it — and why **hathor-core** is built that way.

SUBJECT hathor-core · Part I · Track A (programming concepts)

CHAPTER 01 · Foundations · Concepts

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Classes · Objects · Encapsulation · Inheritance · Polymorphism · Abstraction · ABCs · Composition · Dunder methods

Table of Contents

Chapter 1 — Object-Oriented Programming	3
1.1 The problem: programs that grow	3
1.2 Three ways to organize a program	4
1.3 Classes and objects	6
1.4 Encapsulation — the first pillar	7
1.5 Inheritance — the second pillar	8
1.6 Polymorphism — the third pillar	9
1.7 Abstraction — the fourth pillar	10
1.8 Composition over inheritance	11
1.9 The dunder protocol	12
1.10 Bridge — where this lands in hathor-core	13
Recap	14

Chapter 1 — Object-Oriented Programming

What you'll learn in this chapter

- The problem object-oriented programming exists to solve, and how it compares to the other ways of structuring a program (procedural and functional) — and why a system like a node leans on OOP.
- The four pillars in working order: **encapsulation, inheritance, polymorphism, abstraction** — each motivated, defined, and shown in small, runnable examples.
- The Python machinery that expresses them: classes, `self`, `__init__`, methods, properties, abstract base classes, and the "dunder" protocol methods.
- When **composition** beats inheritance.
- A **bridge** to where each idea reappears in `hathor-core`, so the codebase chapters land on prepared ground.

This is a concept chapter, not a code-tour. There is no `hathor-core` source here on purpose — you will meet the real classes in Part II, and they will make far more sense once these ideas are second nature. Everything below is plain Python you could paste into a shell and run. The final section (§1.10) is the only one that looks forward, pointing at exactly where in the codebase each idea is put to work.

If you already know OOP comfortably, skim §1.2 (the paradigm comparison) and §1.10 (the bridge), and move on.

1.1 The problem: programs that grow

Small programs need almost no structure. A fifty-line script that reads a file, transforms some numbers, and prints a result can be one flat sequence of statements, and you will never get lost in it.

The trouble starts when a program gets big and long-lived. A full node is both: tens of thousands of lines, running for months, juggling many things at once — dozens of network connections, a database, a wallet, a mempool, a mining service. Two specific pains show up as a codebase grows, and OOP is a direct response to both.

Pain 1 — data and the rules about it drift apart. Imagine modelling a bank account as a plain number:

```
balance = 100
# ...hundreds of lines later, in some unrelated function...
balance = balance - 9999      # nothing stopped this; the account is now negative
```

The rule "an account may never go negative" lives only in the programmer's head. The data (`balance`) is just a number sitting in the open, and any line anywhere is free to corrupt it. In a large program, "any line anywhere" means thousands of lines, written by several people over years. The rule will eventually be broken by accident.

What we want is to keep the data and the rules that protect it in the same place, and to forbid the rest of the program from touching the data except through those rules. That bundling is the central idea of OOP.

Pain 2 — many things are almost the same, but not quite. A node has several kinds of vertex (blocks, transactions), several kinds of storage backend (in-memory, on-disk), several kinds of peer-connection state. Each family shares most of its behavior but differs in a few specifics. Without a way to say "these are all accounts, and here is the one bit each kind does differently," you end up copy-pasting near-identical code and then fixing the same bug in five places. We want to write the shared part once and let each variant override only what differs.

Hold those two pains — protect invariants¹ by bundling data with behavior, and share behavior across a family of related types — they are exactly what the pillars in §1.3-§1.7 deliver.

1.2 Three ways to organize a program

OOP is not the only way to structure code, and it is not always the best one. It helps to see it next to its two main alternatives, because real codebases — `hathor-core` included — mix them. Python is a **multi-paradigm**² language: it supports all three, and lets you pick per situation.

Procedural (a.k.a. structured) programming. You organize the program as a set of procedures — functions — that operate on data passed to them. The data and the functions are separate things. This is how most people first learn to code, and it is excellent for straightforward, step-by-step tasks. Its weakness is Pain 1: the data floats free of the rules, and as the program grows, it becomes unclear which functions are allowed to change which data.

```
def deposit(account, amount):      # data (account) and behavior (deposit) are separate
    account["balance"] += amount
```

Functional programming. You build the program out of pure functions³ — functions whose output depends only on their input and which change nothing outside themselves (no side effects⁴) — and you prefer immutable⁵ data that is never modified in place, only transformed into new values. This style shines for data-transformation pipelines and for safe concurrency (if nothing is ever mutated, two threads can't corrupt each other's data). Its awkward spot is modelling things that have identity and change over time — a network connection that opens, exchanges messages, and closes is naturally a stateful entity, and pretending it is a stream of immutable values can be more contortion than clarity.

```
def with_deposit(account, amount):          # returns a NEW account; original untouched
    return {**account, "balance": account["balance"] + amount}
```

Object-oriented programming. You bundle data together with the behavior that operates on it into **objects**, each with its own identity and its own internal **state**⁶. An object guards its own data and exposes a set of operations (methods) as the only sanctioned way to use or change it.

```
class Account:
    def __init__(self, balance):
        self._balance = balance          # the data...
    def deposit(self, amount):
        self._balance += amount          # ...lives with the behavior that guards it
```

Why OOP for a node? A full node is overwhelmingly a collection of long-lived, stateful entities with identity: the manager, each peer connection, the storage layer, the mempool, every vertex with its evolving metadata. Each has a lifecycle (created, started, used, stopped) and rules about what states are valid. That is precisely the shape OOP models best, which is why `hathor-core` uses OOP as its backbone — while still using functional touches for pure data transformation and procedural code for simple scripts. The lesson is not "OOP is best"; it is "match the paradigm to the shape of the problem," and a node's problems are mostly object-shaped.

A note on dogma. You will meet programmers who treat one paradigm as a religion. Resist it. Paradigms are tools; a senior engineer reaches for whichever makes the current code clearest. This book teaches OOP first because `hathor-core` is built on it, not because the others are lesser.

1.3 Classes and objects

Here is the vocabulary, made precise. A **class** is a blueprint for a kind of object. An **object** (or **instance**⁷) is a concrete thing built from that blueprint. The class `Account` describes what every account is and can do; `my_account = Account(100)` builds one specific account. One class, many instances — just as one architectural blueprint yields many houses, each with its own address and contents.

```
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner          # an attribute: data belonging to this instance
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount      # a method: behavior belonging to instances
        return self.balance

alice = Account("Alice", 100)      # instantiation: build an object from the class
bob = Account("Bob")              # balance defaults to 0
alice.deposit(50)                 # call a method on the alice instance
print(alice.balance)              # 150
print(bob.balance)               # 0 - bob is a separate object with its own state
```

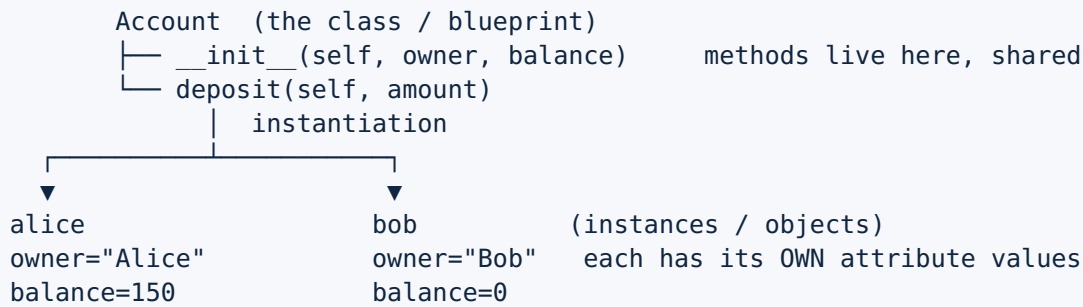
Three pieces of machinery deserve a slow read, because they confuse almost everyone at first:

`__init__` — the initializer. When you write `Account("Alice", 100)`, Python creates a fresh empty object and then immediately calls its `__init__` method to set it up.

`__init__` is where you give the new object its starting **attributes**⁸ (its data). It is not strictly a "constructor"⁹ in the C++/Java sense — the object already exists by the time `__init__` runs — but you can think of it as "the setup routine that runs once, per object, at birth."

`self` — the instance the method is working on. Every method's first parameter is `self`, which is the particular object the method was called on. When you write `alice.deposit(50)`, Python calls `deposit` with `self = alice`. That is how `self.balance += amount` knows to change Alice's balance and not Bob's. `self` is not a keyword — it is just a parameter name (a near-universal convention), and Python passes the instance into it automatically. The single most common beginner stumble is forgetting `self` in a method's signature or when accessing an attribute.

Attributes vs. methods. An attribute is data attached to an object (`alice.owner`, `alice.balance`). A method is a function attached to a class that operates on an instance (`alice.deposit(...)`). Both are reached with the dot. Each instance carries its own attribute values — that is what "its own state" means — while the methods are shared by all instances of the class (they live on the blueprint, not on each house).



1.4 Encapsulation — the first pillar

Encapsulation¹⁰ is the bundling of data with the behavior that operates on it, plus the controlled hiding of that data so the outside world can only touch it through approved operations. It is the direct cure for Pain 1.

Recall the rule "an account may never go negative." With encapsulation, the account enforces its own rule, and no outside code can break it:

```
class Account:
    def __init__(self, balance=0):
        self._balance = balance

    def withdraw(self, amount):
        if amount > self._balance:
            raise ValueError("insufficient funds") # the rule, enforced in one place
        self._balance -= amount

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("amount must be positive")
        self._balance += amount

    @property
    def balance(self): # read-only view of the protected data
        return self._balance
```

Two conventions are doing the work here:

The leading underscore. Naming the attribute `_balance` instead of `balance` is Python's way of saying "this is internal; do not touch it from outside." Python does not forcibly prevent access — unlike some languages, there is no hard `private` keyword — it relies on a shared convention: a leading underscore means "hands off."¹¹ The point is that all changes to the balance now funnel through `deposit` and `withdraw`, where the rule lives. The invariant has exactly one guardian.

The `@property`. The `balance` method decorated with `@property` lets outside code read `account.balance` as if it were a plain attribute, while there is no setter, so it cannot be assigned. `account.balance` works; `account.balance = -5` fails. You get a read-only window onto protected internals. (`@property` is itself an example of a decorator, the subject of Chapter 4 — for now, read it as "turns this method into a smart attribute.")

The payoff: the **invariant** " $\text{balance} \geq 0$ " is now true by construction. You can read every other line of the program and never have to worry that one of them quietly corrupted an account, because the only doors into `_balance` are the two methods that check the rule. That is the whole game of encapsulation — shrink the set of code that can break a rule down to one small, auditable place.

1.5 Inheritance — the second pillar

Inheritance¹² lets a new class take on all the attributes and methods of an existing class and then add to or change them. The existing class is the **superclass** (or base/parent); the new one is the **subclass** (or derived/child). The relationship inheritance models is "**is-a**": a savings account is an account.

This is the cure for Pain 2. Write the shared behavior once in the base class; let each variant add only its differences.

```
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance

    def withdraw(self, amount):
        if amount > self._balance:
            raise ValueError("insufficient funds")
        self._balance -= amount

class SavingsAccount(Account):
    # SavingsAccount IS-A Account
    def __init__(self, owner, balance=0, rate=0.02):
        super().__init__(owner, balance)  # run the base class's setup first
        self.rate = rate                  # then add what's specific to savings

    def add_interest(self):
        # a brand-new method, only on savings
        self._balance += self._balance * self.rate
```

`SavingsAccount` automatically has `withdraw` and `owner` — it did not repeat them. It added `rate` and `add_interest`. Two mechanics matter:

`super().__init__(...)`. A subclass usually wants the base class's setup to run and then its own. `super()` is how a subclass calls up to its parent's version of a method. Here `super().__init__(owner, balance)` runs `Account.__init__` (setting `owner` and `_balance`), after which the subclass sets `rate`. Forgetting the `super()` call is a classic bug: the object skips its parent's initialization and ends up half-built.

Overriding. A subclass can replace a method it inherited by defining one with the same name. This is **method overriding**¹³. Suppose savings accounts forbid overdrawing below a minimum:

```
class SavingsAccount(Account):
    MIN_BALANCE = 100

    def withdraw(self, amount):
        # overrides Account.withdraw
        if self._balance - amount < self.MIN_BALANCE:
            raise ValueError("would drop below minimum balance")
        super().withdraw(amount)
        # reuse the parent's logic for the rest
```

Notice the override can still call `super().withdraw(...)` to reuse the parent's work and only add its extra check. That is the ideal: specialize the difference, inherit the rest.

A caution. Inheritance is tempting and easy to over-use. A deep tower of subclasses ($A \rightarrow B \rightarrow C \rightarrow D$, each changing a little) becomes hard to follow, because to understand class D you must hold all four in your head at once. §1.8 gives the usual remedy. A good rule: use inheritance for genuine "is-a" relationships, and be suspicious of hierarchies more than two or three deep.

1.6 Polymorphism — the third pillar

Polymorphism¹⁴ (Greek: "many shapes") means that code written against a general type works, unchanged, on any of its specific variants — each variant supplying its own behavior behind a shared name. You write `account.withdraw(x)` once, and it does the right thing whether `account` is a plain `Account` or a `SavingsAccount` with its extra rule.

```
def end_of_day(accounts, fee=5):
    for account in accounts:
        account.withdraw(fee)
        # same call; each object runs ITS OWN withdraw

portfolio = [Account("Alice", 200), SavingsAccount("Bob", 500)]
end_of_day(portfolio)
# Alice runs Account.withdraw; Bob runs SavingsAccount.withdraw — automatically.
```

`end_of_day` knows nothing about savings accounts. It will keep working when you add `CheckingAccount`, `BusinessAccount`, and ten more types years later, as long as each has a `withdraw` method. This is why polymorphism matters in a long-lived codebase: it lets you add new variants without editing the code that uses them. New behavior, no rewrites.

Duck typing. Python takes polymorphism a step further than many languages. It does not require the objects to share a common base class at all — it only requires them to have the method you call. The slogan is "if it walks like a duck and quacks like a duck, treat it as a duck." This is **duck typing**¹⁵ :

```
class Report:
    def withdraw(self, amount):
        print(f"deducting {amount} from the report budget")

end_of_day([Account("Alice", 200), Report()]) # works! Report just needs .withdraw
```

`Report` is not an account and inherits nothing from `Account`, but because it has a `withdraw` method, `end_of_day` accepts it. Python checks for the method at the moment of the call, not in advance. This is flexible and concise; the trade-off is that the "contract" (what methods an object must have) is implicit. The next pillar makes that contract explicit when you want it to be.

1.7 Abstraction — the fourth pillar

Abstraction¹⁶ is the practice of describing what something does while hiding how. A car's steering wheel is an abstraction: you turn it to steer, without knowing about the rack-and-pinion underneath. In code, abstraction lets you program against a contract — a set of operations something promises to provide — rather than against any one concrete implementation.

The tool for stating such a contract explicitly is the **abstract base class (ABC)**¹⁷. An ABC declares the methods that every concrete subclass must provide, but does not (necessarily) implement them. It cannot be instantiated directly — it is a contract, not a thing.

A motivating example you will meet for real in the storage chapters: suppose the node needs to store data, and you want to support more than one backend — an in-memory store for tests, a file-backed store for production — interchangeably.

```

from abc import ABC, abstractmethod

class Storage(ABC):
    @abstractmethod
    def save(self, key, value): ...
    # the contract
    # "every Storage MUST provide save"

    @abstractmethod
    def load(self, key): ...
    # "...and load"

class MemoryStorage(Storage):
    def __init__(self):
        self._data = {}
    def save(self, key, value):
        self._data[key] = value
    def load(self, key):
        return self._data.get(key)
    # one concrete implementation

class FileStorage(Storage):
    def save(self, key, value):
        ... # write to disk
    def load(self, key):
        ... # read from disk
    # another, with the same contract

```

Now the rest of the program can be written against `Storage` — "give me something I can `save` to and `load` from" — and neither know nor care which concrete backend it received:

```

def remember(store: Storage, key, value):
    store.save(key, value)
    # works with MemoryStorage, FileStorage, or any future o

```

Two benefits fall out. First, `Storage()` raises an error — Python refuses to instantiate an abstract class, so you cannot accidentally use the empty contract as if it were real storage. Second, if you write a new backend and forget to implement `load`, Python refuses to instantiate that too, telling you the contract is unmet. The implicit duck-typing contract of §1.6 is now written down and enforced.

This pairing — an abstract interface plus interchangeable concrete implementations — is one of the most common shapes in `hathor-core`, and recognizing it is half of reading the codebase.

1.8 Composition over inheritance

Inheritance ("is-a") is not the only way to build a class out of other classes. The alternative is **composition**¹⁸: an object contains other objects and delegates work to them. The relationship is "**has-a**": an account has a transaction log.

```

class TransactionLog:                                # a small, focused object
    def __init__(self):
        self._entries = []
    def record(self, text):
        self._entries.append(text)

class Account:
    def __init__(self, balance=0):
        self._balance = balance
        self._log = TransactionLog()                 # HAS-A log (composition)
    def deposit(self, amount):
        self._balance += amount
        self._log.record(f"deposit {amount}")        # delegate logging to the log object

```

`Account` did not inherit from `TransactionLog` — a log is not a kind of account — it holds one and hands it the logging job. Why prefer this? Composition keeps each object small and single-purpose, and you can swap the contained object (a `FileLog`, a `NullLog` that records nothing) without touching the account. The widely-repeated advice "**favor composition over inheritance**" means: reach for "has-a" by default, and use "is-a" inheritance only when one type is genuinely a specialized kind of another. Composition avoids the deep, brittle towers warned about in §1.5.

A quick test when you are unsure which to use: say it out loud. "A savings account is an account" — inheritance fits. "An account is a transaction log" — nonsense; it has one — composition fits.

1.9 The dunder protocol

Python lets your objects plug into the language's own syntax through specially-named methods wrapped in double underscores — **dunder**¹⁹ methods ("double underscore"), also called magic methods. You have already met `__init__`. A few more you will see constantly:

```

class Money:
    def __init__(self, cents):
        self.cents = cents

    def __repr__(self):
        return f"Money({self.cents}c)"  # how the object prints in logs/debugger

    def __eq__(self, other):
        return self.cents == other.cents  # what == means for two Money objects

    def __add__(self, other):
        return Money(self.cents + other.cents)  # what + means

a = Money(150)
b = Money(150)
print(a)          # Money(150c)          ← __repr__ ran
print(a == b)     # True                  ← __eq__ ran (without it, == compares identity)
print(a + b)      # Money(300c)         ← __add__ ran

```

The pattern: by implementing the protocol method, you teach Python how your object should behave with built-in operations — printing, `==`, `+`, `len()`, iteration, indexing, and more. This is itself a form of polymorphism: `len(x)` works on a list, a string, or your object, because each implements `__len__`. You rarely need many of these, but recognizing them matters — when you see `__repr__` or `__eq__` in `hathor-core`'s classes, now you know they are defining how those objects print and compare, not some obscure ritual.

1.10 Bridge — where this lands in `hathor-core`

You will not need any of these terms in the abstract for long; Part II is built almost entirely from them. So that the codebase chapters land on prepared ground, here is the map from pillar to place. These are forward-pointers, not explanations — each gets its full treatment in the chapter named.

Bridge — OOP in the codebase (full treatment in the chapters named):

- **Inheritance & a base class.** The node's entire data model is one inheritance hierarchy: a `BaseTransaction` superclass with `Block`, `Transaction`, and `MergeMinedBlock` subclasses, each overriding the parts that differ. This is the §1.5 pattern at the heart of the ledger — **Chapter 25**.
- **Abstraction & ABCs.** Storage is defined as an abstract contract with interchangeable backends (in-memory for tests, RocksDB for production) — exactly the `Storage` ABC of §1.7 — **Chapters 27-28**. Hathor also uses a dedicated interface library (`zope.interface`) on top of plain ABCs; that wrinkle is covered where it appears — **Chapter 16 & 34**.
- **Polymorphism.** The verification and consensus code calls shared methods on whatever vertex it is handed, each type supplying its own behavior — §1.6 in action — **Chapters 31-32**.
- **Encapsulation & invariants.** A vertex's `metadata` guards rules like "weight only grows"; the manager guards the node's lifecycle state. The §1.4 "one guardian per invariant" idea is everywhere — **Chapters 25 & 29**.
- **Composition.** The `HathorManager` does not inherit its abilities; it holds a storage object, a connections manager, a wallet, and so on, delegating to each — the §1.8 "has-a" pattern at the largest scale — **Chapter 29**.
- **Dunder methods.** Vertices implement `__eq__` / `__hash__` so they can be compared and used as dictionary keys; many classes implement `__repr__` for logging — **Chapters 25 onward**.

There is also a related concept this chapter deliberately left out: the **design patterns** (factory, builder, observer, ...) that are named, reusable arrangements of the objects we just built. Those are Chapter 3 — read this chapter as the grammar and Chapter 3 as the idioms.

Recap

Pillar / idea	One-line definition	Cures	Python machinery
Class / object	Blueprint vs. concrete instance built from it	—	<code>class</code> , <code>__init__</code> , <code>self</code>
Encapsulation	Bundle data with its rules; hide the data	data corrupted from afar (Pain 1)	<code>_underscore</code> , <code>@property</code>
Inheritance	A subclass reuses & specializes a base class	duplicated near-identical code (Pain 2)	<code>class Sub(Base)</code> , <code>super()</code> , <code>override</code>

Pillar / idea	One-line definition	Cures	Python machinery
Polymorphism	One call, each type runs its own version	editing callers when adding variants	shared method names, duck typing
Abstraction	Program to a contract, hide the how	implicit, unenforced contracts	<code>ABC</code> , <code>@abstractmethod</code>
Composition	An object has others and delegates	brittle deep inheritance towers	hold an instance as an attribute
Dunder protocol	Teach objects to use built-in syntax	—	<code>__repr__</code> , <code>__eq__</code> , <code>__add__</code> , ...

Object-oriented programming earns its place in a node because a node is, at bottom, a population of long-lived stateful things that must each protect their own rules and come in related families. Encapsulation keeps each thing's rules in one auditable place; inheritance and polymorphism let related things share behavior and vary only where they must; abstraction lets the rest of the program depend on contracts rather than concretions; composition keeps the whole assembly from collapsing into a brittle tower. Keep the \$1.10 bridge in mind — when Chapter 25 opens `BaseTransaction` and you see a base class with overriding subclasses, encapsulated metadata, and dunder methods, you will be reading a sentence in a grammar you already know. The next chapter takes on the one concept that turns this static picture into a running node: how a single-threaded program does many things at once, through callbacks and the event loop.

1. An invariant is a condition that must always hold true for an object to be valid — e.g. "an account's balance is never negative." Much of good design is about identifying invariants and ensuring no code can violate them. ↩
2. A multi-paradigm language supports several programming styles (procedural, functional, object-oriented) rather than forcing one. Python, JavaScript, and Scala are multi-paradigm; older languages often were not. ↩
3. A pure function always returns the same output for the same input and has no observable effect beyond that return value. `len(x)` is pure; a function that prints, or edits a global, is not. ↩
4. A side effect is any change a function makes to the world outside itself — modifying a global variable, writing a file, printing, mutating an argument. Pure functional code minimizes side effects. ↩
5. Immutable data cannot be changed after creation; you make a modified copy instead. Python strings and tuples are immutable; lists and dicts are mutable (changeable in place). ↩
6. State is the data an object currently holds — its attribute values at a moment in time. An object with state "remembers" things between method calls; that memory is what makes it stateful. ↩
7. An instance is one concrete object built from a class. "Instantiate" means "create an instance." `Account("Alice")` instantiates the `Account` class. ↩

8. An attribute is a named piece of data attached to an object (e.g. `account.balance`) or to a class. Reached with the dot operator. ↩
9. A constructor is the routine that creates and sets up a new object. In Python the setup step is `__init__`; technically object creation is handled by `__new__`, which you almost never need to touch. ↩
10. Encapsulation = bundling data with the methods that operate on it, and restricting outside access to that data so it can only change through approved methods. ↩
11. Python has no enforced `private`. Convention: one leading underscore (`_x`) means "internal, please don't touch"; two leading underscores (`__x`) trigger name mangling, a stronger but still bypassable form of hiding. The community relies on discipline over locks. ↩
12. Inheritance lets a subclass acquire the attributes and methods of a superclass, then add or override behavior. Models an "is-a" relationship. ↩
13. Overriding is redefining, in a subclass, a method that already exists in the superclass, so instances of the subclass use the new version. Distinct from overloading (multiple methods with the same name but different parameters), which Python does not do in the classic sense. ↩
14. Polymorphism lets a single piece of code operate on objects of different types, each responding to the same method name in its own way. ↩
15. Duck typing is Python's style of polymorphism: an object's suitability is determined by whether it has the needed methods/attributes, not by what class it inherits from. "If it quacks, it's a duck." ↩
16. Abstraction = exposing what an operation does while hiding how it is done, so callers depend on a simple contract rather than complex internals. ↩
17. An abstract base class (ABC) declares methods that subclasses must implement and cannot itself be instantiated. Python provides it via the `abc` module (`ABC`, `@abstractmethod`). It turns an implicit duck-typing contract into an explicit, enforced one. ↩
18. Composition builds an object by giving it other objects as attributes and delegating work to them — a "has-a" relationship, as opposed to inheritance's "is-a." ↩
19. Dunder = "double underscore." Dunder methods like `__init__`, `__repr__`, `__eq__`, `__len__` let your objects integrate with Python's built-in syntax and functions. Also called magic or special methods. ↩